



KTU NOTES

The learning companion.

**KTU STUDY MATERIALS | SYLLABUS | LIVE
NOTIFICATIONS | SOLVED QUESTION PAPERS**

🌐 Website: www.ktunotes.in

MODULE-3

SYLLABUS

Interrupt & interrupt handling - Hardware and Software interrupts.

I/O and memory interfacing – Address decoding– Interfacing I/O ports -Programmable Peripheral Interface PPI 8255 - Modes of operation- Interfacing of seven segment LED.

Introduction to embedded systems, Current trends and challenges, Applications of embedded systems- Hard and soft real time systems.

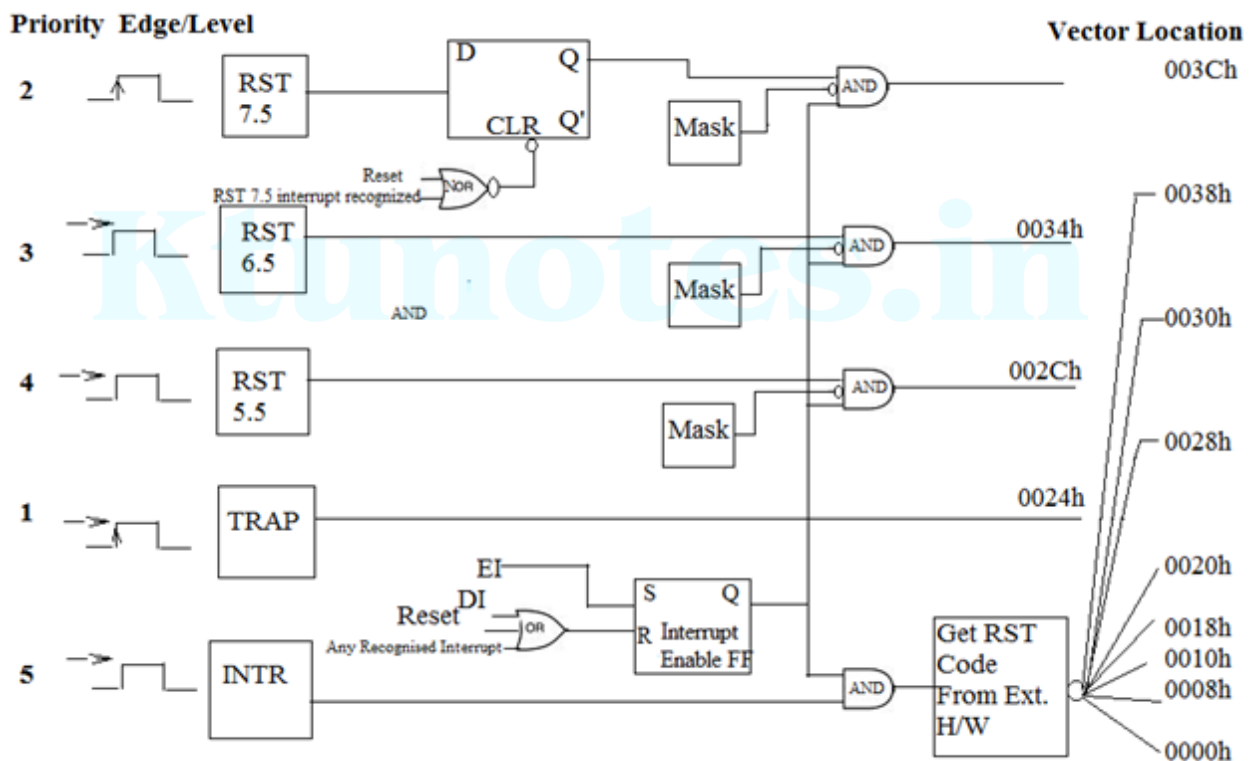
Introduction to microcontrollers- Microprocessor Vs Microcontroller- 8051 Microcontrollers – Hardware - Microcontroller architecture and programming model - I/O port structure - Register organization -General purpose RAM - Bit addressable RAM - Special Function Registers (SFRs).

3.1 Interrupt Structure in 8085

- Interrupt is signal send by an external device to the processor, to request the processor to perform a particular task or work.

3.1.1 Interrupt Process in 8085:

- The processor will check the interrupts always at the last machine cycle of each instruction.
- If there is any interrupt, it accepts the interrupt and sends the $\overline{INTA}$ signal to the peripheral.
- The current value of program counter will be saved to stack and starting address (vector address) of interrupt service routine (ISR) will be copied to program counter.
- The processor executes the interrupt service routine (ISR).
- It returned to main program by last instruction of ISR (RET) after retrieving the address stored at the stack.



Interrupt Structure of 8085

3.1.2 Types of Interrupts:

It supports two types of interrupts.

- **Hardware**
- **Software**

- The software interrupts are program instructions. These instructions are inserted at desired locations in a program. While running a program, If a software interrupt instruction is encountered, then the processor executes an interrupt service routine (ISR)
- The 8085 has eight software interrupts from RST 0 to RST 7. The vector address for these interrupts can be calculated as follows.
- $\text{Interrupt number} * 8 = \text{vector address}$
- For eg:- for RST1, $1 * 8 = 08 = 08H$ So, Vector address for interrupt RST 5 is 0008H.

Hardware interrupts:

- An external device initiates the hardware interrupts by applying an appropriate signal at the interrupt pin of the processor.
- If the interrupt is accepted then the processor executes an interrupt service routine.
- The 8085 has five hardware interrupts
(1) TRAP (2) RST 7.5 (3) RST 6.5 (4) RST 5.5 (5) INTR

1. TRAP:

- This interrupt is a non-maskable interrupt. It is unaffected by any mask or interrupt enable.
- TRAP is a vectored interrupt.
- TRAP has the highest priority.
- TRAP interrupt is edge and level triggered.

2. RST 7.5:

- The RST 7.5 interrupt is a maskable interrupt.
- It has the second highest priority.
- It is edge sensitive.
- Maskable interrupt. It can be disabled by DI instruction.
- Enabled by EI instruction.

3. RST 6.5 and 5.5:

- The RST 6.5 and RST 5.5 both are level triggered.
- RST 6.5 and RST 5.5 are maskable interrupts. These can be disabled by DI instruction.
- The RST 6.5 has the third priority whereas RST 5.5 has the fourth priority.

4. INTR:

- INTR is a maskable interrupt. It can be disabled by DI instruction.
- It can be enabled by EI instruction.
- Non- vectored interrupt. After receiving INTA signal, it has to supply the address of ISR.
- It has lowest priority.
- It is a level sensitive interrupts.

The following sequence of events occurs when INTR signal goes high.

1. The 8085 checks the status of INTR signal during execution of each instruction.
 2. If INTR signal is high, then 8085 complete its current instruction and sends an interrupt acknowledge signal INTA to the device that interrupted.
 3. In response to the acknowledge signal, interrupted device places an RST instruction on the data bus.
 4. On receiving the instruction, the 8085 save the address of next instruction on stack and execute received instruction.
 5. Then the microprocessor executes the interrupt service routine.
-

3.1.3 Priority of Interrupts:

The 8085 microprocessor has five interrupt inputs. They are TRAP, RST 7.5, RST 6.5, RST 5.5, and INTR. These interrupts have a fixed priority of interrupt service. If two or more interrupts go high at the same time, the 8085 will service them on priority basis. The TRAP has the highest priority followed by RST 7.5, RST 6.5, RST 5.5 and INTR. The priority of interrupts in 8085 is shown in the table.

TRAP - 1
RST 7.5 - 2
RST 6.5 - 3
RST 5.5 - 4
INTR - 5

3.1.4 Maskable and Non-Maskable Interrupts:

Maskable Interrupt:

An Interrupt that can be disabled or ignored by the instructions of CPU is called as Maskable Interrupt.

Eg: RST 7.5, RST 6.5, RST 5.5 and INTR are maskable Interrupts.

Non-Maskable Interrupt: An interrupt that cannot be disabled or ignored by the instructions of CPU is called as Non-Maskable Interrupt.

Eg: TRAP interrupt of 8085

3.1.5 Vectored and Non-Vectored Interrupts:

Vectored interrupt

In vectored interrupts, the processor automatically branches to the specific address in response to an interrupt.

Eg: TRAP, RST 7.5, RST 6.5, RST 5.5

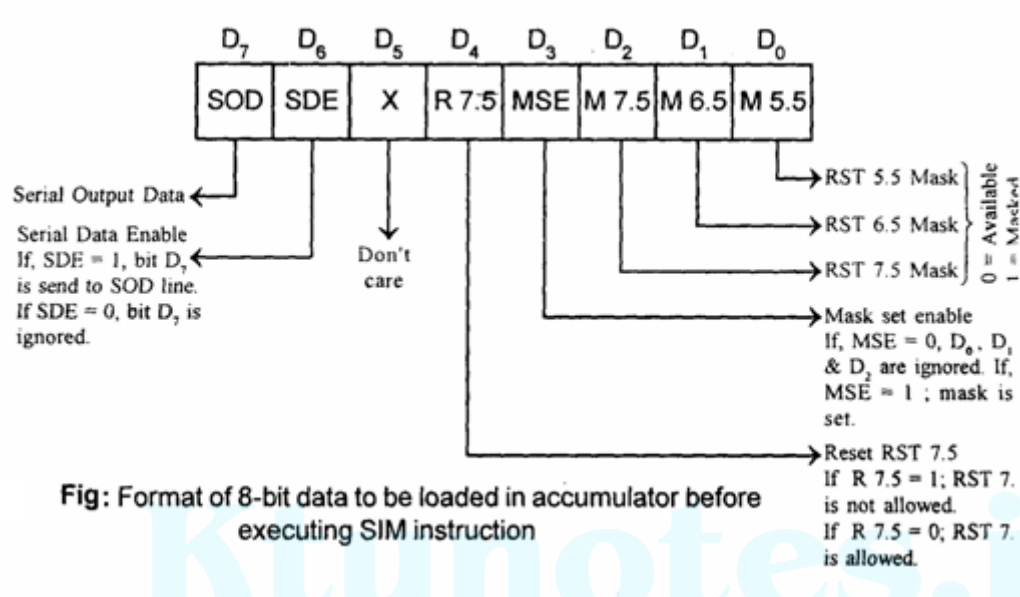
Non-vectored interrupt

But in non -vectored interrupts the interrupted device should give the address of the interrupt service routine (ISR).

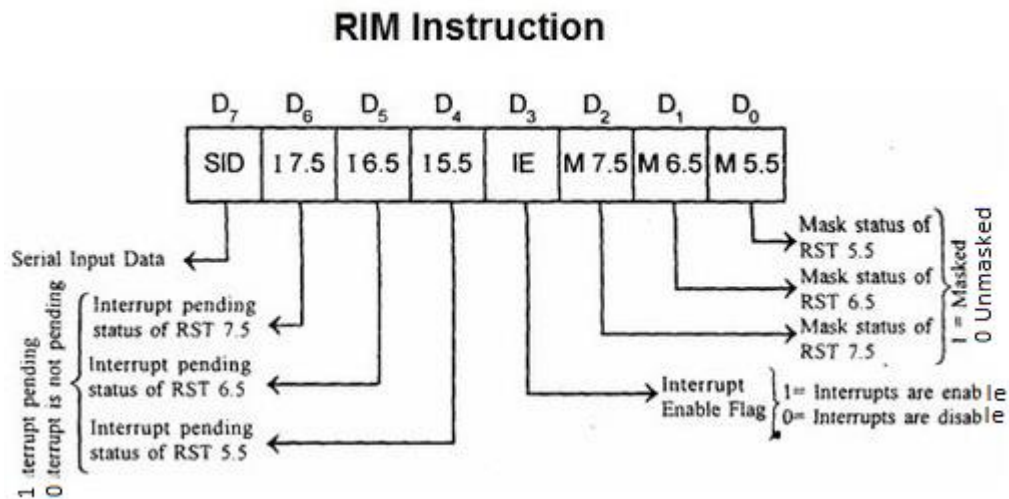
Eg: INTR

SIM and RIM for interrupts:

- The 8085 provide additional masking facility for RST 7.5, RST 6.5 and RST 5.5 using SIM instruction.
- The status of these interrupts can be read by executing RIM instruction.
- The masking or unmasking of RST 7.5, RST 6.5 and RST 5.5 interrupts can be performed by moving an 8-bit data to accumulator and then executing SIM instruction.
- The format of the 8-bit data is shown below.



- The status of pending interrupts can be read from accumulator after executing RIM instruction.
- When RIM instruction is executed an 8-bit data is loaded in accumulator, which can be interpreted as shown in fig.



- Interfacing is the process of connecting devices together such that they can exchange information.
- There are various communication devices like keyboard, printer etc. So we need to interface these devices with microprocessor by using latches and buffers. This type of interfacing is known as I/O interfacing.

I/O Interfacing Techniques:

I/O devices can be interfaced with microprocessor by two ways:

- Memory mapped I/O and
- I/O mapped I/O or isolated I/O.

Memory mapped IO

- With memory mapped I/O there is a **single address space** for memory and I/O devices as shown in figure.
- The processor treats the I/O devices just like memory locations and **uses same instructions to access both memory and I/O devices**.
- **Single read line and single write line** are required on the bus to access both memory and I/O devices.

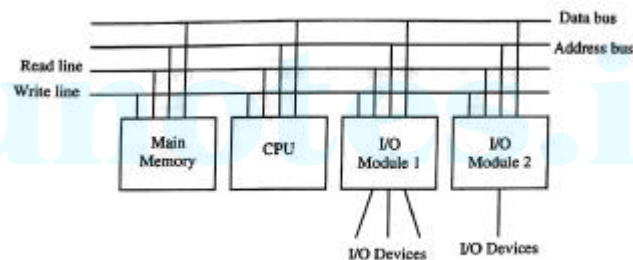


Figure 3.2: Structure of Memory Mapped I/O

I/O mapped I/O or isolated I/O:

- With I/O mapped I/O, the memory and I/O devices are **addressed separately**.
- Processor **uses different instructions for memory and I/O devices**. (ie, MOV for memory and IN,OUT for I/O)
- **Different read and write control signals** are used for memory and I/O devices as shown in the figure below.

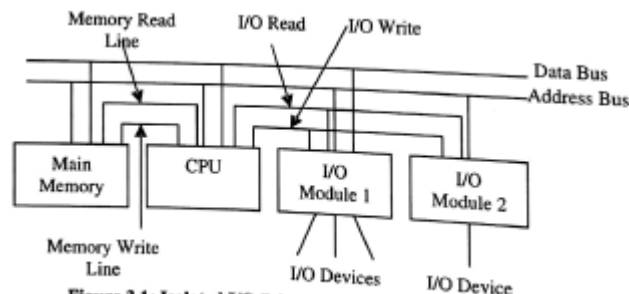


Figure 3.1: Isolated I/O Scheme or I/O Mapped I/O

- The process of interconnecting memory with microprocessor through buses and other hardware components is known as **memory interfacing**.

Address Decoding:

Address decoding is the way by which microprocessor decodes an address to select a memory location among the total available memory locations. Two types of address decoding techniques are there.

- Absolute Decoding or Full Decoding
- Partial Decoding or Linear Decoding

Absolute Decoding or Full Decoding:

- The decoding in which **all available address lines are used** for decoding to select a location is called absolute decoding technique.
- The figure below shows the absolute decoding technique.

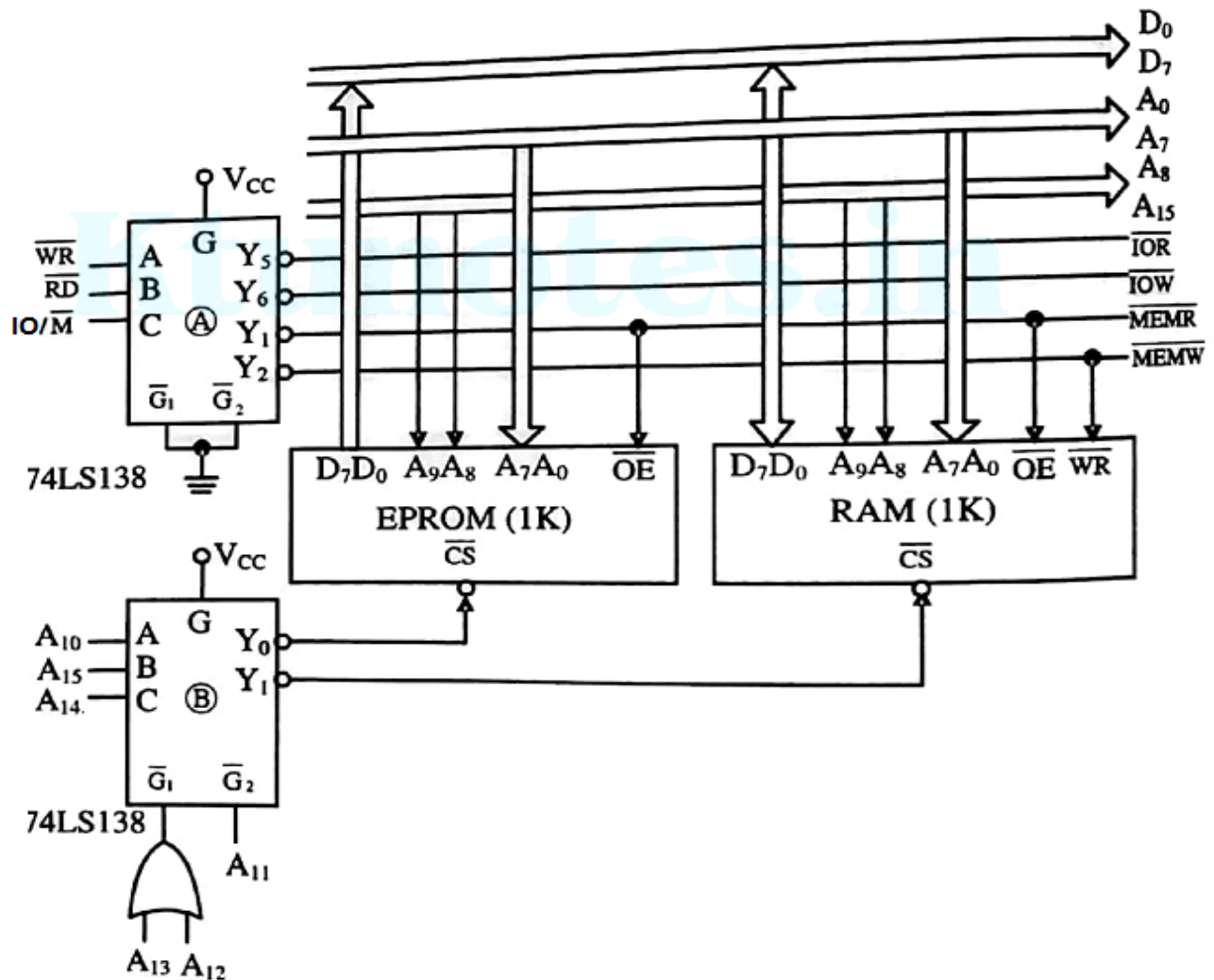
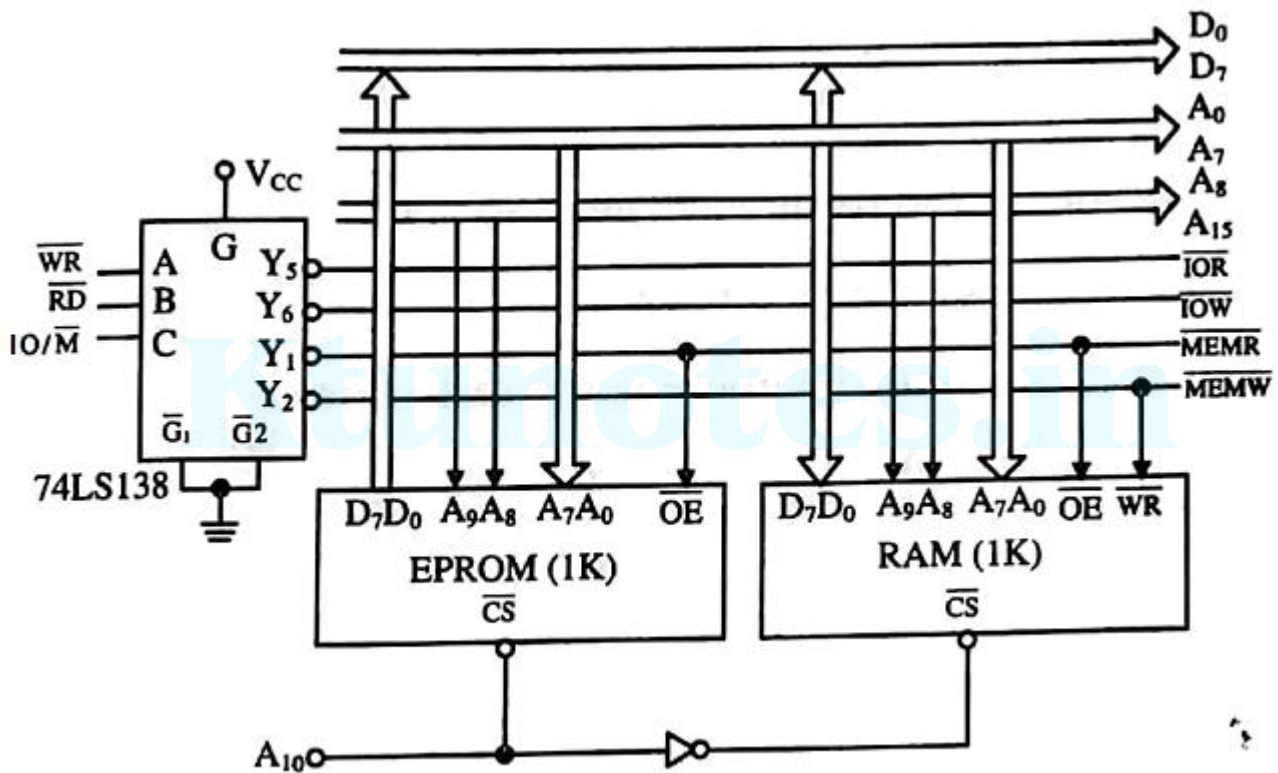


Figure 3.6: Absolute Decoding Technique

- This decoding technique is normally used in large memory systems.

[illegible]

- The decoding in which **all available address lines are not used** for decoding to select a location is called partial decoding technique.
- The figure below shows the partial decoding technique.



- In small systems, the hardware for decoding circuit can be eliminated by using individual high order address lines to select memory chips. This is the partial decoding technique.
- The cost of the decoding circuit can be reduced with this technique.
- Drawback is the possibility of having multiple addresses.
- The memory map for partial decoding technique is shown below.

[illegible]

Full Address Decoding	Partial Address decoding
1. All higher address lines are decoded to select the memory or I/O device.	1. Few higher address lines are decoded to select the memory or I/O device.
2. More hardware is required to design decoding logic.	2. Hardware required to design decoding logic is less and sometimes it can be eliminated.
3. Higher cost for decoding circuit.	3. Less cost for decoding circuit.
4. No Multiple addresses.	4. It has a advantage of multiple addresses.
5. Used in large systems	5. Used in small systems

----- UNIVERSITY MODEL QUESTION -----

Memory Interfacing Problems: (For more problems, refer class notes)

Consider a system in which 32KB memory space is implemented using four numbers of 8KB memory. Interface the EPROM and RAM with 8085 processor. (16KB RAM and 16KB EROM)

Ans:

--The total memory capacity is 32KB, hence 15 address lines are required to address it. (A0-A14)

--Since available memory chips are 8KB, 13 address lines are required to address each chip. (A0-A12)

--So remaining 2 lines A13 and A14 can be used to select the memory chip.

a) Using Absolute Decoding Technique:

The interfacing steps are shown below:

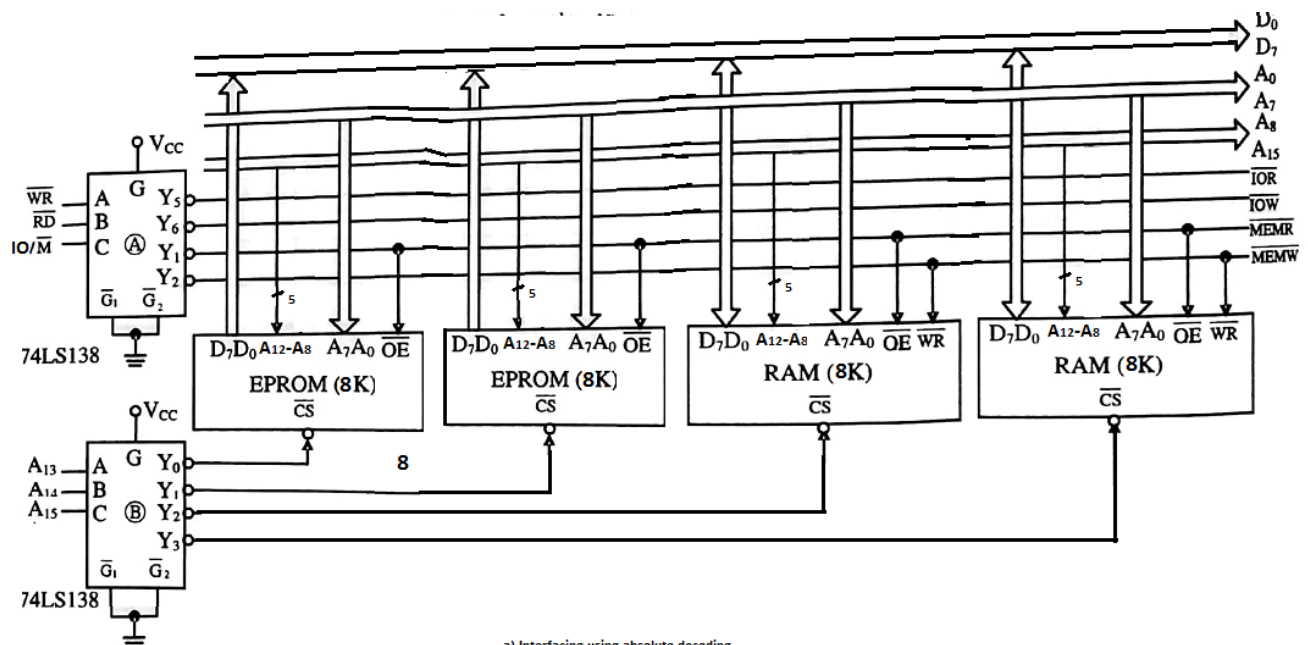
Step 1) Each 8KB memory requires 13 address lines and so the address lines A₀-A₁₂ of the processor are connected to 13 address pins of all the memory.

Step 2) Use a 3 to 8 decoder (74LS138) and connect A₁₃ and A₁₄ to the first two inputs of decoder (inputs A and B)

Step 3) These four chip select signals can be used to select one of the four memory IC at any one time.

Step 4) Since the decoding technique is absolute, A₁₅ is connected as third input to decoder and make it 0 always.

Step 5) The simplified schematic memory organisation is shown.



a) Interfacing using absolute decoding

Memory ICs	A ₁₅	A ₁₄	A ₁₃	A ₁₂	A ₁₁	A ₁₀	A ₉	A ₈	A ₇	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀	Address
Starting address of EPROM1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0000H
End address of EPROM1	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1FFFH
Starting address of EPROM2	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	2000H
End address of EPROM2	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	3FFFH
Starting address of RAM1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	4000H
End address of RAM1	0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	5FFFH
Starting address of RAM2	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	6000H
End address of RAM2	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	7FFFH

Consider a system in which 32KB memory space is implemented using four numbers of 8KB memory. Interface the EPROM and RAM with 8085 processor.

b) Using Partial Decoding Technique:

The interfacing steps are shown below:

Step 1) Each 8KB memory requires 13 address lines and so the address lines A₀-A₁₂ of the processor are connected to 13 address pins of all the memory.

Step 2) The address lines A₁₃-A₁₄ can be decoded using a 2-to-4 decoder to generate four chip select signals.

Step 3) These four chip select signals can be used to select one of the four memory IC at any one time.

Step 4) Since the decoding is partial, remaining address line (A₁₅) is not used.

Step 5) The simplified schematic memory organisation is shown.

3.4 INTERFACING I/O PORTS

3.4.1 PROGRAMMABLE PERIPHERAL INTERFACE (PPI)-8255

Features:

- It is a programmable device.
- It has 24 I/O programmable pins.

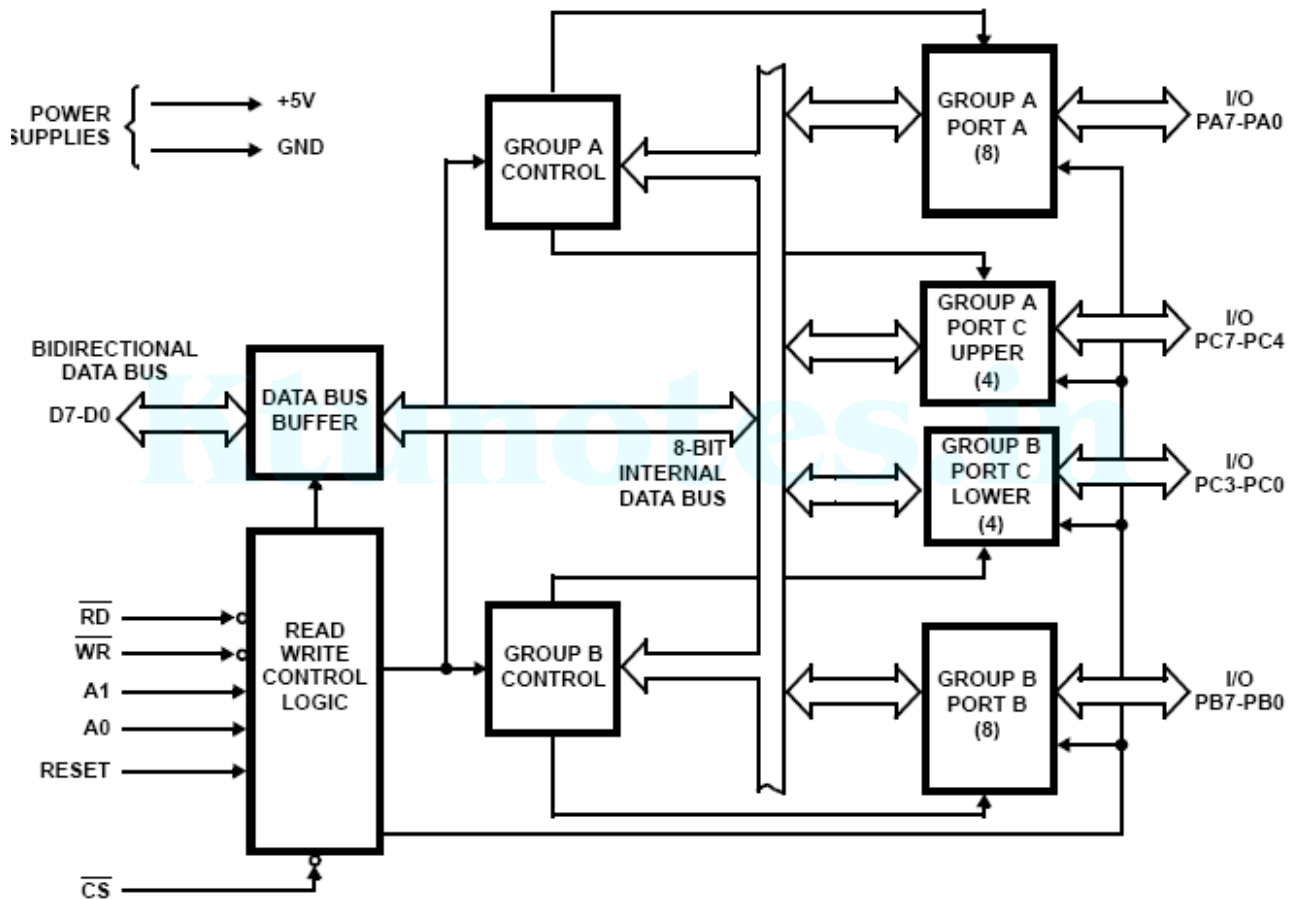
Function of pins:

- Data bus(D₀-D₇):These are 8-bit bi-directional buses, connected to 8086 data bus for transferring data.
- CS: This is Active Low signal. When it is low, then data is transfer from 8085.
- Read: This is Active Low signal, when it is Low read operation will be start.
- Write: This is Active Low signal, when it is Low Write operation will be start.
- Address (A₀-A₁):This is used to select the ports. like this

A1	A0	Select
0	0	PA
0	1	PB
1	0	PC
1	1	Control reg.

- PA₀-PA₇: It is the 8-bit bi-directional I/O pins used to send the data to peripheral or to receive the data from peripheral.
- PB₀-PB₇: Similar to PA
- PC₀-PC₇: This is also 8-bit bidirectional I/O pins. These lines are divided into two groups.
 1. PC₀ to PC₃ (Lower Groups)
 2. PC₄ to PC₇ (Higher groups)
 These two groups can be worked separately.

Block Diagram:



Data Bus buffer:

- It is a 8-bit bidirectional Data bus.
- Used to interface between 8255 data bus with system bus.
- The internal data bus and Outer pins D₀-D₇ pins are connected in internally.
- The direction of data buffer is decided by Read/Control Logic.

Read/Write Control Logic:

- This is getting the input signals from control bus and Address bus
- Control signal are RD and WR.
- Address signals are A₀, A₁, and CS.

Group A and Group B control:

- Group A and B get the Control Signal from CPU and send the command to the individual control blocks.
- Group A send the control signal to port A and Port C (Upper) PC7-PC4.
- Group B send the control signal to port B and Port C (Lower) PC3-PC0.

PORT A:

- This is a 8-bit buffered I/O latch.
- It can be programmed by mode 0 , mode 1, mode 2 .

PORT B:

- This is a 8-bit buffer I/O latch.
- It can be programmed by mode 0 and mode 1.

PORT C:

- This is a 8-bit Unlatched buffer Input and an Output latch.
- It is splitted into two parts, Port C Upper and Port C Lower.
- It can be programmed by bit set/reset operation.

Modes of Operation :

- Basically, there are two modes, BSR mode and I/O mode.

6. BIT SET/RESET MODE:

- Any one of the 8-bits of PORT C can be Set or Reset depending upon the select bits on control word register.

7. I/O MODE:

3 types of I/O modes are there, mode0, mode1 and mode2.

MODE 0(Simple input / Output):

- In this mode , two 8 bit ports(port A and port B) and two 4 bit ports(port C upper and port C lower) are available for I/O operations.
- Features:
- Any port can be used as an input or output port.
- Ports do not have Handshake or interrupt capability.

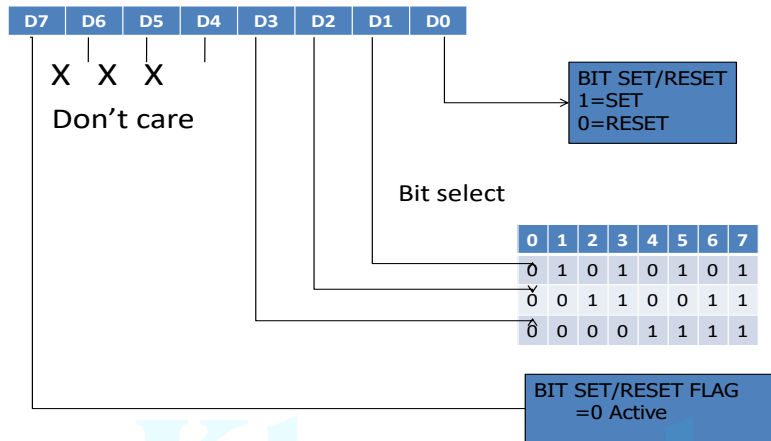
MODE 1 :(Strobed I/O mode)

- Two groups-group A and group B are available for strobed data transfer.
- Each group contains one 8-bit data I/O port and one 4 bit control port.
- The 8 bit data port can be either used as input or output port.
- Out of 8 bit port C,PC3-PC5 are used to generate control signals for port A and PC0-PC2 are used to generate control signals for port B. The lines PC6 and PC7 may be used as independent I/O lines.

- This mode allows bidirectional data transfer over a single 8-bit data bus (port A) using handshake signals.
- Port A is working as 8-bit bidirectional.
- 5 bit control port PC3-PC7 is used for generating/accepting handshaking signals of port A.
- Here, port B and three lines of port C (PC2-PC0) may be used in either simple I/O mode or strobed mode.

Control word register format.

For BSR **mode**:



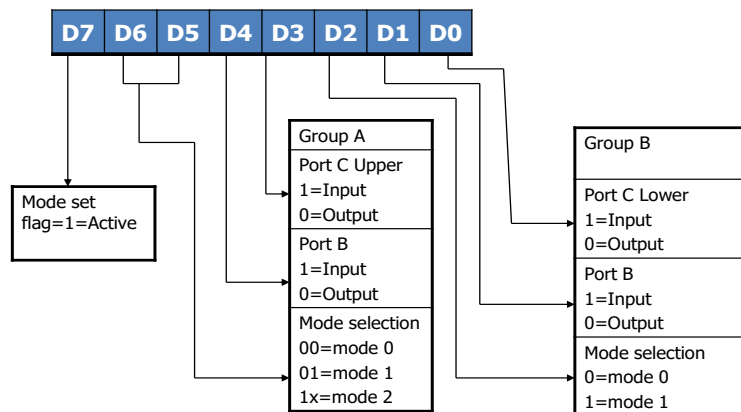
- PC0-PC7 is set or reset as per the status of D0.

Example:

- To Set PC3, control register will be 0XXX0111.
- To reset PC4, control register will be 0XXX01000.
- X is a don't care.

• FOR I/O MODE:

The mode format for I/O as shown in figure



Answer: D7 D6 D5 D4 D3 D2 D1 D0

1 0 0 0 1 0 1 1

UNIVERSITY QUESTION

What are handshaking signals?

Handshaking signals are dedicated signals to coordinate data transfer between two devices with different speed. The following are the commonly used handshaking signals:

STB (Strobe Input) :

This signal is generated by a peripheral device that it has transmitted a byte of data.

IBF (Input buffer full) :

This signal is an acknowledgement by the 8255 to indicate that the input latch has received the data byte.

IOBF(Output Buffer Full) :

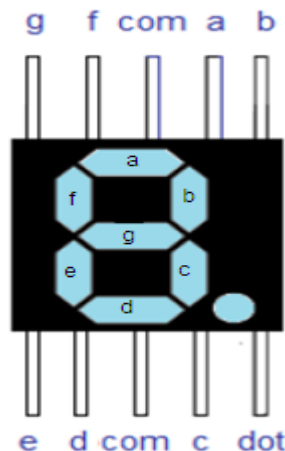
This is an output signal that goes low when the microprocessor writes data into the output latch of the 8255.

ACK (Acknowledge) :

This is an input signal from a peripheral that must output a low when the peripheral receives the data from the 8255 ports.

3.4.2 INTERFACING SEVEN SEGMENT DISPLAY WITH 8051

- A seven segment display consists of seven LEDs arranged in the form of a squarish '8'.
- Different characters can be displayed by selectively glowing the required LED segments.
- Seven segment displays are of two types, **common cathode** and **common anode**.
- In common cathode type , the cathode of all LEDs are tied together to a single terminal which is usually labeled as 'com' and the anode of all LEDs are left alone as individual pins labeled as a, b, c, d, e, f, g & h (or dot) .
- In common anode type, the anode of all LEDs are tied together as a single terminal and cathodes are left alone as individual pins.



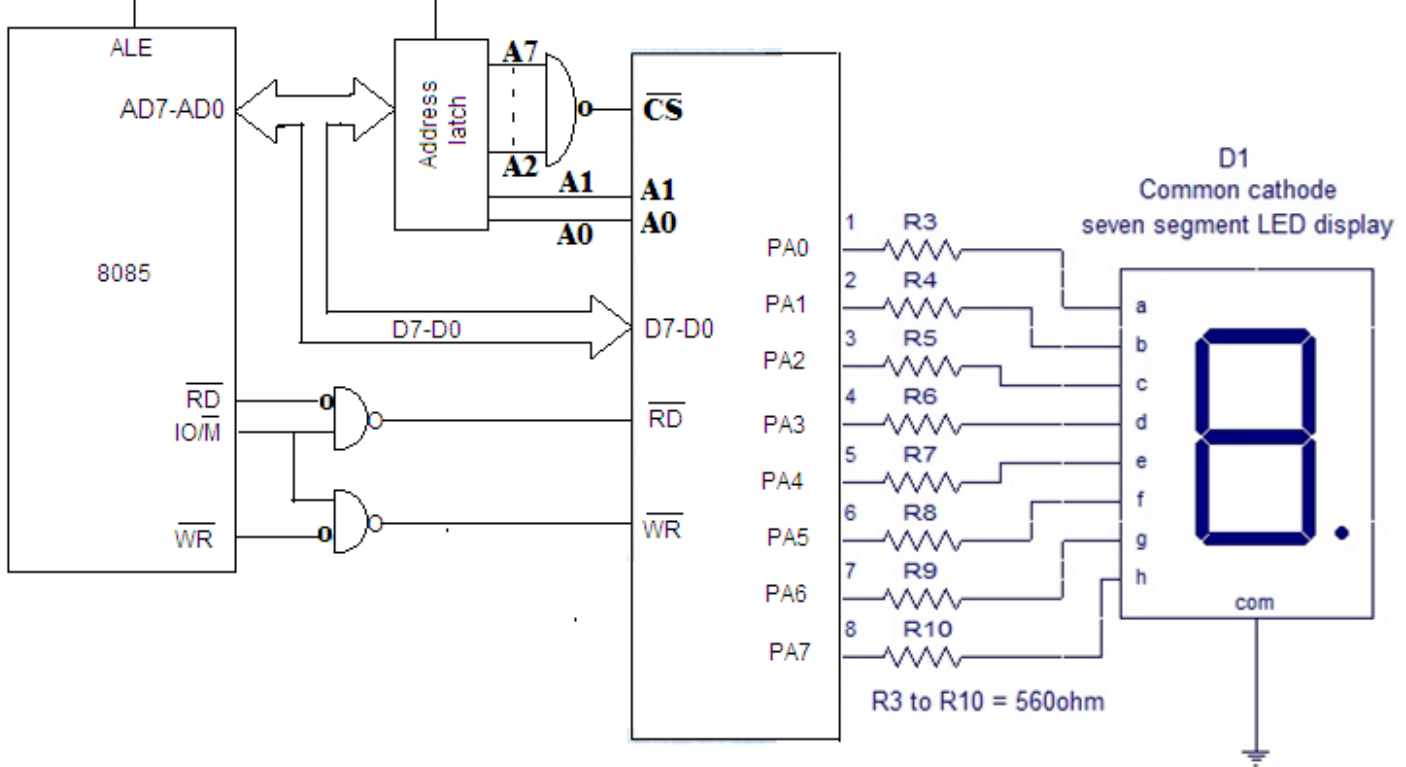


Figure: Interfacing seven segment display with 8085

- Seven segment LED display is interfaced with 8085 using 8255 PPI.
- 8255 PPI is interfaced using address lines, data lines and control line RD and WR.
- Seven segment LED display is connected to any one of the port by configuring the port in simple IO mode.
- Seven segment LED display used here is a common cathode type, hence When logic 1 is send to any pin of Port A, the LED will conduct the current and it will glow.
- When logic 0 is send to any pin of Port A, then the LEDs will be off.

Following are the 8 bit values to display various digits for a common cathode type seven segment display.

Digit	Dp	g	f	e	d	c	b	a	Hex val
0	0	0	1	1	1	1	1	1	3F
1	0	0	0	0	0	1	1	0	06
2	0	1	0	1	1	0	1	1	5B
3	0	1	0	0	1	1	1	1	4F
4	0	1	1	0	0	1	1	0	66
5	0	1	1	0	1	1	0	1	6D
6	0	1	1	1	1	1	0	1	7D
7	0	0	0	0	0	1	1	1	07
8	0	1	1	1	1	1	1	1	7F
9	0	1	1	0	0	1	1	1	67

The 8255 control word is written as follows:

D7 D6 D5 D4 D3 D2 D1 D0

1 0 0 0 0 0 0 0

- The required ALP is as follows:

```
                MVI A, 80                ; initialize 8255 as output port in simple IO mode.
                OUT CWR
AGAIN: MVI A, 3FH
                OUT Port A
                CALL DELAY
                MVI A, 06H
                OUT Port A
                CALL DELAY
                MVI A, 5BH
                OUT Port A
                CALL DELAY
                MVI A, 4FH
                OUT Port A
                CALL DELAY
                MVI A, 66H
                OUT Port A
                CALL DELAY
                MVI A, 6DH
                OUT Port A
                CALL DELAY
                MVI A, 7D
                OUT Port A
                CALL DELAY
                MVI A, 07
                OUT Port A
                CALL DELAY
                MVI A, 7F
                OUT Port A
                CALL DELAY
                MVI A, 67
                OUT Port A
                CALL DELAY
                JMP AGAIN ; repeat
```

```
DELAY:         MVI C, FFH ; delay routine
BACK:          DCR C
                NOP
                NOP
                JNZ BACK
                RET
```

- An embedded system is an electronic system, which includes a single chip microcomputers (Microcontrollers) like the 8051 or ARM.
- It is configured to perform a specific dedicated application.
- Software is programmed into the on chip ROM of the single chip computer. This software is not accessible to the user, and software solves only a limited range of problems.
- Here the microcomputer is embedded or hidden inside the system.
- Every **embedded microcomputer** system, accepts inputs, performs computations, and generates outputs and runs in “real time.”

(For Example a typical automobile now a days contains an average of ten microcontrollers. In fact, modern houses may contain as many as 150 microcontrollers and on average a consumer now interacts with microcontrollers up to 300 times a day. General areas that employ embedded systems covers every branch of day to day science and technology, namely Communications, automotive, military, medical, consumer, machine control etc...

Ex: Cell phone, Digital camera, Microwave Oven, MP3 player, Portable digital assistant & automobile antilock brake system etc.)

3.5.1 Recent trends in Embedded systems

With the fast developments in semiconductor industry and VLSI technology, one can find tremendous changes in the embedded system design in terms of processor speed, power, communication interfaces including network capabilities and software developments like operating systems and programming languages etc.

Processor speed and Power :

- With the advancements in processor technology, the embedded systems are now days designed with 16,32 bit processors which can work in real time environment.
- These processors are able to perform high speed signal processing activities which resulted in the development of high definition communication devices like 3G mobiles etc.
- Also the recent developments in VLSI technology has paved the way for low power battery operated devices which are very handy and have high longevity.
- Also, the present day embedded systems are provided with higher memory capabilities, so that most of them are based on tiny operating systems like android.

Communication interfaces :

- Most of the present day embedded systems are aimed at internet based applications.
- So, the communication interfaces like Ethernet, USB, wireless LAN etc. have become very common resources in almost all the embedded systems.
- The developments in memory technologies also helped in porting the TCP/IP protocol stack and

between any two devices any where in the globe.

Operating systems :

- With recent software developments ,there is a considerable growth in the availability of operating systems for embedded systems.
- Mainly new operating systems are developed which can be used in real time applications.
- There are both commercial RTOSes like Vx Works , QNX,WIN-CE and open source RTOSes like RTLinux etc.
- The Android OS in mobiles has revolutionized the embedded industry.

Programming Languages :

- There is also a remarkable development in the programming languages.
- Languages like C++, Java etc. are now widely used in embedded application programming. For example by having the Java virtual machine in a mobile phones ,one can download Java applets from a server and can be executed on your mobile.
- In addition to these developments, now a days we also find new devices like ASICs and FPGAs in the embedded system market. These new hardware devices are popular as programmable devices and reconfigurable devices.

3.5.2 Challenges in Embedded Systems

1. Amount and type of hardware needed.
 - Optimizing various hardware elements for a particular design.
2. Taking into account the design metrics
 - Design metrics examples –power dissipation, physical size, number of gates and engineering, prototype development and manufacturing costs.
3. Optimizing the Power Dissipation.
 - Clock Rate Reduction and Operating Voltage Reduction
4. Disable use of certain structural units of the processor to reduce power dissipation the processor to reduce power dissipation.
 - Control of power requirement, for example, by screen auto-brightness control
5. Process Deadlines
 - Meeting the deadline of all processes in the system while keeping the memory, power dissipation, processor clock rate and cost at minimum is a challenge.
6. Flexibility and Upgradeability
 - Ability to offer the different versions of a product for marketing and offering the product in advanced versions later on.
7. Reliability
 - Designing reliable product by appropriate design and thorough testing, verification and validation is a challenge.
8. Testing, Verification and Validation
 - Testing – to find errors and to validate that the implemented software is as per the specifications and requirements to get reliable product.

3.5.3 Application Areas of Embedded Systems:

- The embedded systems have a huge variety of application domains which varies from very low cost to very high cost and from daily life consumer electronics to industry automation equipments, from entertainment devices to academic equipments, and from medical instruments to aerospace and weapon control systems.
- So, the embedded systems span all aspects of our modern life. The following table gives the various applications of embedded systems.

S.No	Embedded System	Application
1	Home Appliances	Dishwasher, washing machine, microwave, Top-set box, security system , HVAC system, DVD, answering machine, garden sprinkler systems etc..
2	Office Automation	Fax, copy machine, smart phone system, modern, scanner, printers.
3	Security	Face recognition, finger recognition, eye recognition, building security system , airport security system, alarm system.
4	Academia	Smart board, smart room, OCR, calculator, smart cord.
5	Instrumentation	Signal generator, signal processor, power supplier, Process instrumentation,
6	Telecommunication	Router, hub, cellular phone, IP phone, web camera
7	Automobile	Fuel injection controller, anti-locking brake system, air- bag system, GPS, cruise control.
8	Entertainment	MP3, video game, Mind Storm, smart toy.
9	Aerospace	Navigation system, automatic landing system, flight attitude controller, space explorer, space robotics.
10	Industrial automation	Assembly line, data collection system, monitoring systems on pressure, voltage, current, temperature, hazard detecting system, industrial robot.
11	Personal	PDA, iPhone, palmtop, data organizer.
12	Medical	CT scanner, ECG , EEG , EMG ,MRI, Glucose monitor, blood pressure monitor, medical diagnostic device.
13	Miscellaneous:	Elevators, tread mill, smart card, security door etc.

3.5.4 Real-time embedded systems

- An embedded system which gives the required output in a specified time or which strictly follows the time dead lines for completion of a task is known as a Real time system.
- i.e a Real Time system , in addition to functional correctness, also satisfies the time constraints .
- There are two types of Real time systems.(i) Soft real time system and (ii) Hard real time system.

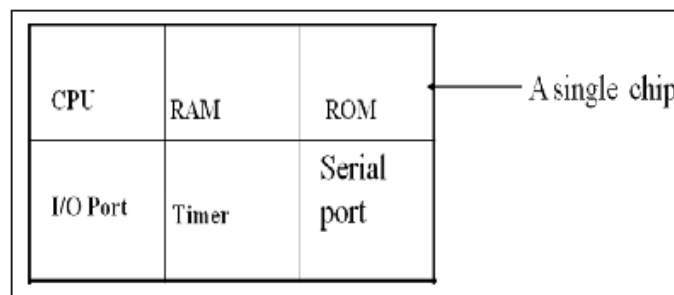
- A Real time system in which ,the violation of time constraints will cause only the degraded quality, but the system can continue to operate is known as a Soft real time system.
- In soft real-time systems, the design focus is to offer a guaranteed bandwidth to each real-time task and to distribute the resources to the tasks.
- Ex: A Microwave Oven , washing machine ,TV remote etc.

2. **Hard Real-Time system :**

- A Real time system in which ,the violation of time constraints will cause critical failure and loss of life or property damage or catastrophe is known as a Hard Real time system.
- These systems usually interact directly with physical hardware instead of through a human being
- Ex: Deadline in a missile control embedded system , Delayed alarm during a Gas leakage , car airbag control system , A delayed response in pacemakers ,Failure in RADAR functioning etc

3.6 MICROCONTROLLER

- A microcontroller is a programmable digital processor with necessary peripherals.
- The design has all the features found in microprocessor CPU: ALU,PC, SP and registers. It also has other features needed to make a complete computer: ROM, RAM, Parallel I/O, serial I/O, Counters and clock circuits.
- Block diagram of a typical Microcontroller which is a true computer on a chip is shown in fig 3.1
- The prime use of microcontroller is to control the operation of a machine using a fixed program that is stored in ROM and that does not change over the lifetime of the system.



3.6.1 Microprocessors versus Microcontrollers:

<u>MICROPROCESSORS</u>	<u>MICROCONTROLLERS</u>
Contains ALU, general purpose registers, stack pointer, program counter, clock timing circuit and interrupt circuit.	Contains the circuitry of microprocessor and in addition it has built in ROM, RAM, I/O devices, timers and counters.
Have many instructions to move data between memory and CPU.	Have one or two instructions.
Have one or two bit handling instructions.	Have many instructions for handling the memory.
Access times for memory and I/O devices are more.	Less access times for built in memory and I/O devices.
Microprocessor based systems require more hardware.	Microcontroller based system require less hardware, reducing the PCB size and increasing the reliability.
They are more flexible in design point of view.	Less flexible from the design point of view.
Has single memory map for data and code.	Has separate memory map for data and code.
Less number of pins is multifunctional.	More number of pins are multifunctional.

3.6.2 8051 MICROCONTROLLER

Features of 8051

- (1). 8 bit microcontroller originally developed by Intel in 1980.
- (2). Highperformance CMOS Technology.
- (3). Contains Total 40 pins.
- (4). Address bus is of 16 bit & data bus is of 8 bit.
- (5). 4K bytes internal ROM (program).
- (6). 128 bytes internal RAM (data).
- (7). Four 8 bit I/O ports.
- (8). Two 16 bit timers.
- (9). Serial interface Communication.
- (10). 64K external code & data memory space.
- (11). 210 bit addressable locations.
- (12). Internal memory consists of on chip ROM and on chip data RAM.
- (13). 8051 implements a separate memory space for programs (code) and data.
- (14). Operating frequency is 24MHz-33MHz.
- (15). +5V Regulated DC power supply is required to operate .
- (16). It has four 8 bit ports, total 32 I/O lines.
- (17). RAM, ROM, I/O ports, one serial port and timers are all on-chip.
- (18). 6 interrupts (2 are external with 2 priority levels).
- (19). Low power Idle and Power down Modes.
- (20). Full duplex UART.
- (21). 8051 has 21 special function registers (SFRs)

3.6.3 ARCHITECTURE OF 8051 MICROCONTROLLER

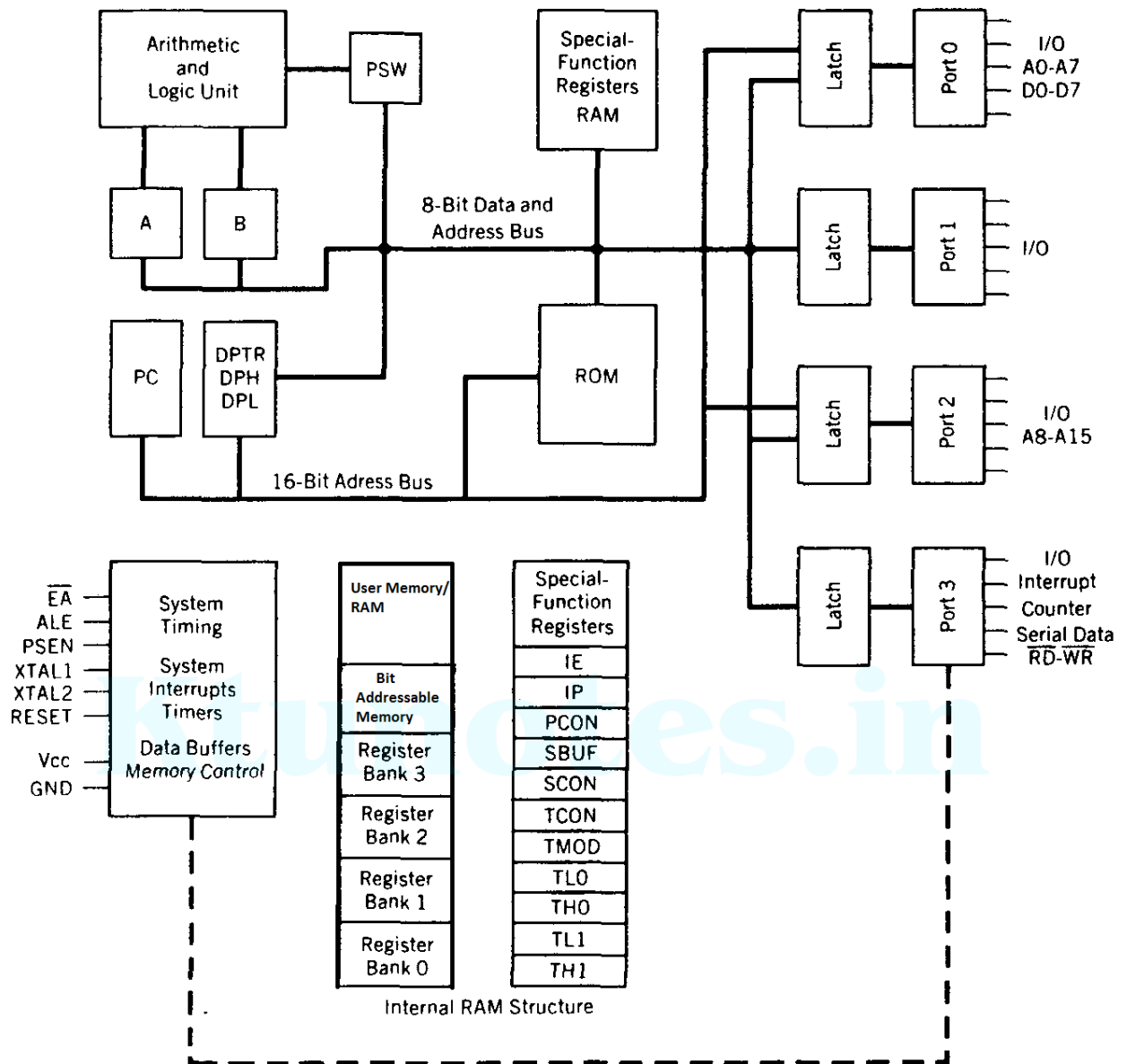


Fig. 5.2 ARCHITECTURE OF 8051

Description:

1. ALU:

- It performs the arithmetic operations such as addition, subtraction, multiplication and division.
- The unit can perform logical operations such as AND, OR; and Exclusive-OR, as well as rotate, clear, and complement.
- The ALU can also manipulate one bit as well as eight-bit data types.
- Individual bits may be set, cleared, complemented, tested, and used in logic computation.

2. Accumulator:

- It is an 8-bit register.
- It holds a data for arithmetic and logical operations.
- Results of arithmetic and logical operations are stored in accumulator.

3. B register:

- An 8-bit general-purpose register.
- Used for direct multiplication and division along with accumulator

4. Program Status Word:

- Many instructions implicitly or explicitly affect (or are affected by) several status flags, which are grouped together to form the Program Status Word (Flag register).
- It also used to select the memory bank.

B ₇	B ₆	B ₅	B ₄	B ₃	B ₂	B ₁	B ₀
CY	AC	F0	RS1	RS0	OV	-	P

Program Status Word

CY, the carry flag

- This flag is set whenever there is a carry out from the D7 bit.

AC, the auxiliary carry flag

- If there is a carry from D3 to D4 during an ADD or SUB operation, this bit is set; otherwise, it is cleared.

P, the parity flag

- The parity flag reflects the number of 1 s in the A (accumulator) register only. If the A register contains an odd number of 1s, then P = 1. Therefore, P = 0 if A has an even number of 1s.

OV, the overflow flag

- This flag is set whenever the result of a signed number operation is too large.

F0, User flag

- This flag is user defined flag. Programmer can use this flag as per the requirements.

RS1 and RS0 are used to select the register bank as follows:

RS1	RS0	BANK SELECTION	
0	0	00H - 07H	Bank 0
0	1	08H - 0FH	Bank 1
1	0	10H - 17H	Bank 2
1	1	18H - 1FH	Bank 3

5. Stack Pointer (SP):

- The stack pointer register is 8 bit wide.
- It is incremented before data is stored during PUSH and CALL instructions.
- After reset the value of SP is 07H.

6. Data Pointer (DPTR):

- The data pointer is a 16-bit register used to hold the 16-bit address of external data memory.
- This can also be used as two numbers of 8-bit data pointer namely DPH and DPL.
- The 8-bit data pointers are used for accessing internal RAM and SFR.
- The 16-bit data pointer is used for accessing external data memory.

7. Program Counter (PC):

- It is a 16 bit register.
- PC is used, as address pointer to access program instructions and it is automatically incremented after every byte of instruction fetch.

8. I/O Ports:

- The 8051 has four numbers of 8-bit ports namely port-0, port- 1, port-2 and port-3.
- Each port has a latch and driver (or buffer).
- When external memory is employed the port-0 lines will function as multiplexed low byte address/data lines and port-2 lines will function as high byte address lines.
- Also the port pins P3.7 and P3.6 are used to output read and write control signals respectively.
- The port-1 is dedicated I/O port and does not have any alternate function.

9. Instruction Register (IR) & timing and control unit:

- The controller will fetch the instructions one by one, starting from the address stored in PC and store in Instruction Register, which decodes the instructions and give information to timing and control unit.
- Using the information supplied by the IR unit the control signals necessary for internal and external operations are generated by the timing and control unit.

10. Timer/Counter:

- The 8031/8051 has two 16-bit programmable timer/counter namely timer-1 and timer 0.
- In the counter mode of operation, they can count the number of high to low transitions of the signal applied to the timer pins.
- In timer mode of operation, they can be independently programmed to work in any one of the four operation modes. They are called mode-0, mode-1, mode-2 and mode-3.

11. Serial port:

- The 8031/8051 has a serial data communication circuit that uses pins RXD (P3.0) and TXD (P3.0) for connecting serial communication device.
- Special function registers related to serial port are SBUF, SCON and PCON.

12. Special Function Registers:

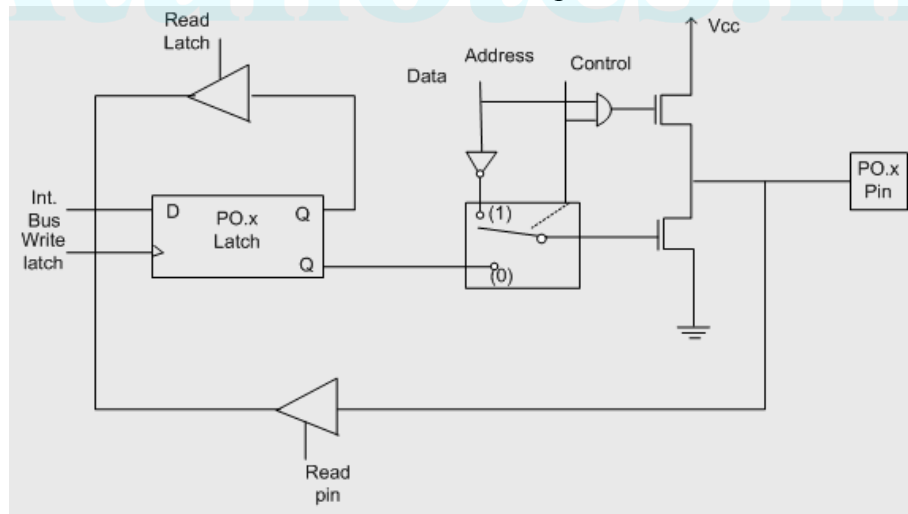
- Special function registers are registers with dedicated functions.

3.6.4 I/O PORTS

- 8051 has 32 I/O pins configured as four 8 bit parallel ports ($P_0 - P_3$).
- All 4 ports are bidirectional, can be configured as either input or output or both.
- All port pins are multi functional except pins of Port 1.
- Each port consists of latch, output driver and input buffer.

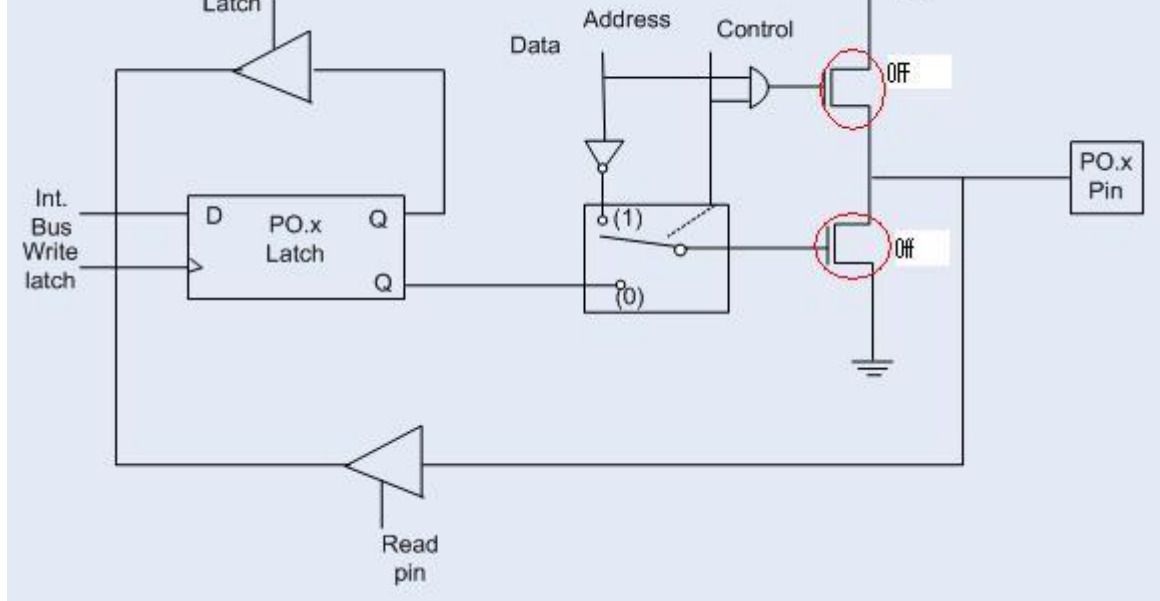
PORT0:

- Port -0 has 8 pins identified as P0.0-P0.7.
- The structure of a Port-0 pin is shown below.
- Port-0 can be configured as a normal bidirectional I/O port or it can be used for address/data interfacing for accessing external memory.
- When control is '1', the port is used for address/data interfacing. When the control is '0', the port can be used as a normal bidirectional I/O port.



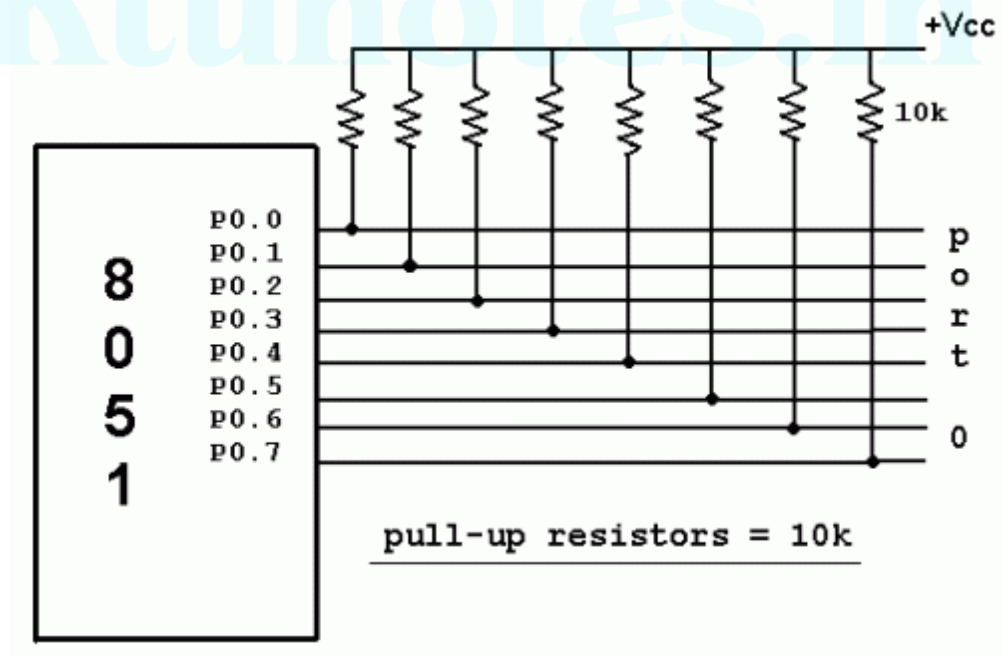
PORT 0 as an Input Port

Let us assume that control is '0'. When the port is used as an input port, '1' is written to the latch. In this situation both the output MOSFETs are 'off'. Hence the output pin has floats hence whatever data written on pin is directly read by read pin.

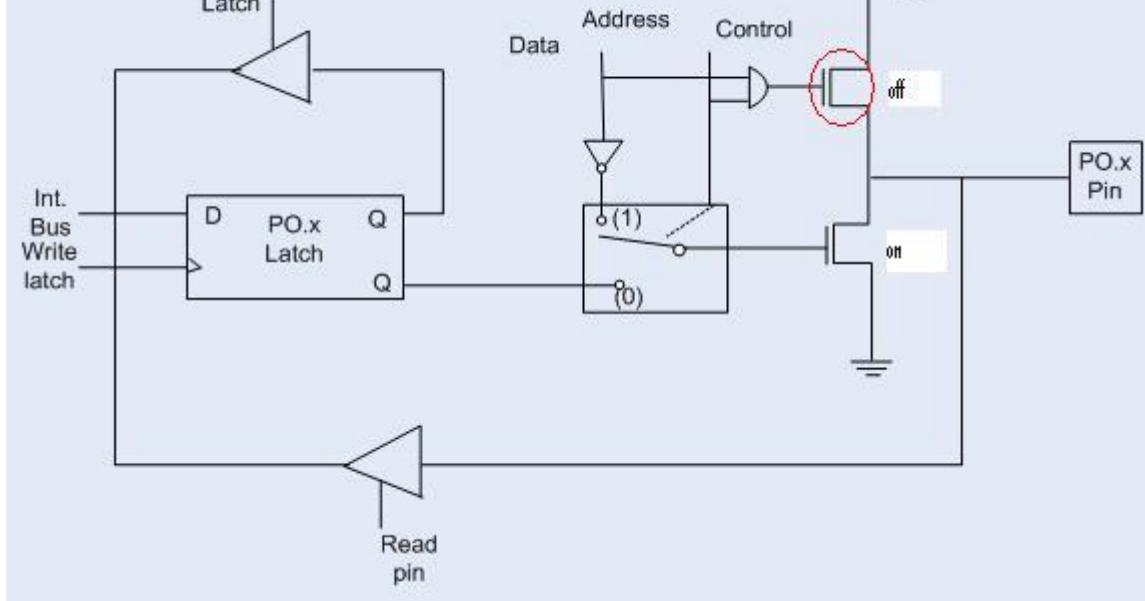


PORT 0 as an Output Port

Suppose we want to write 1 on pin of Port 0, a '1' written to the latch which turns 'off' the lower FET while due to '0' control signal upper FET also turns off as shown in fig. above. Here we want logic '1' on pin but we are getting a floating value. So to convert that floating value into logic '1' we need to connect the pull-up resistor in parallel to the upper FET. This is the reason **why we needed to connect pull-up resistor to port 0 when we want to initialize port 0 as an output port.**

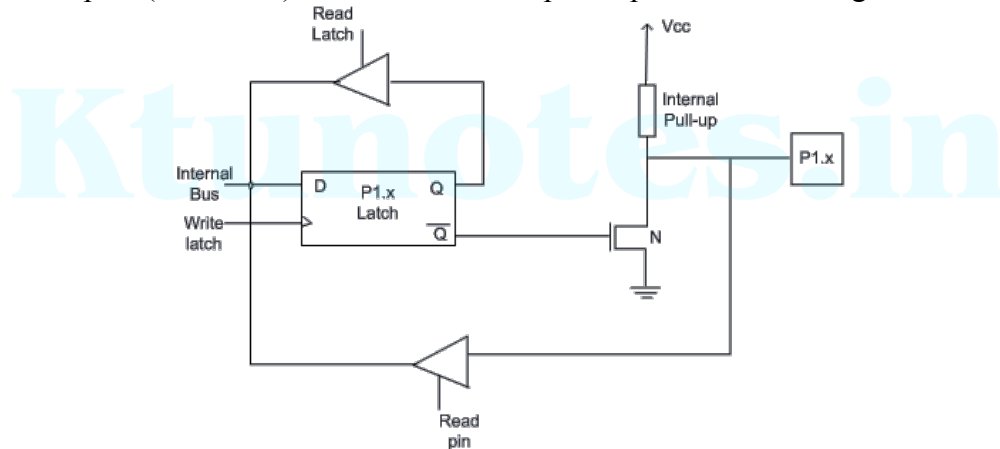


If we want to write '0' on pin of port 0, when '0' is written to the latch, the pin is pulled down by the lower FET. Hence the output becomes zero.



PORT-1

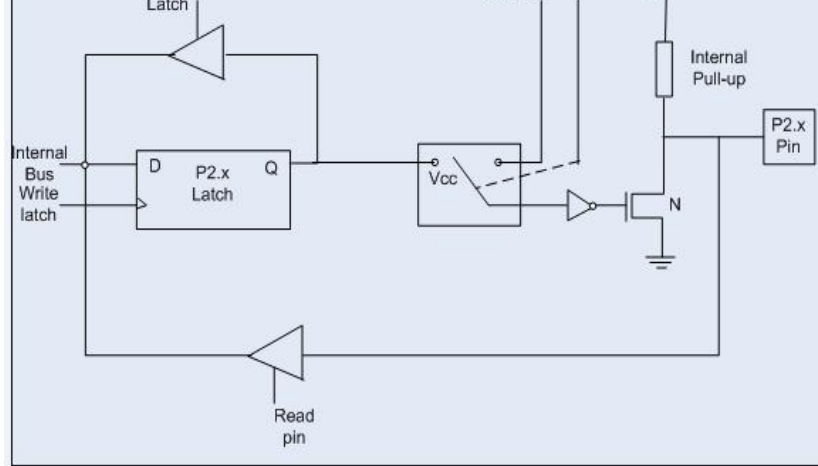
- Port-1 has 8 pins (P1.1-P1.7). The structure of a port-1 pin is shown in fig



- Port-1 does not have any alternate function i.e. it is dedicated solely for I/O interfacing.
- When used as output port, the pin is pulled up or down through internal pull-up.
- To use port-1 as input port, '1' has to be written to the latch. In this input mode the bit written to the pin by the external device can be read the processor.

PORT-2

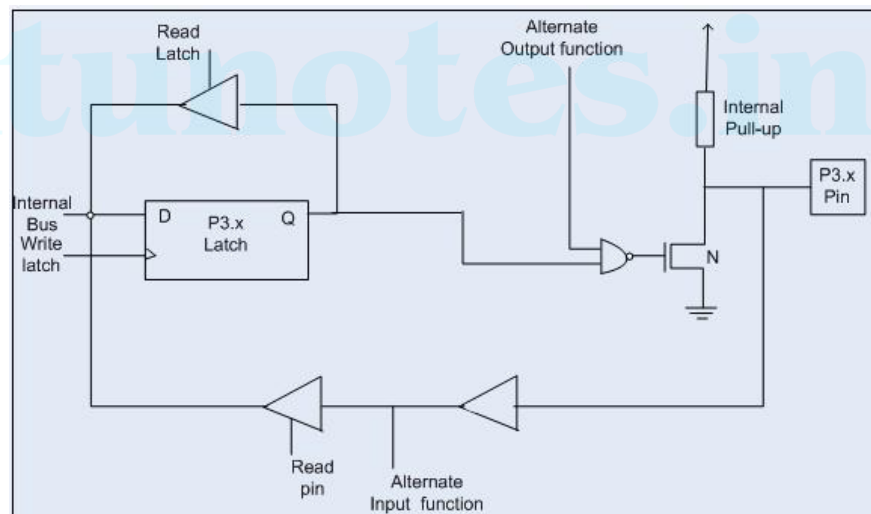
The structure of a port-2 pin is shown in fig. below. It has 8-pins (P2.0-P2.7).



- Port-2 is used for higher external address byte or a normal input/output port.
- The I/O operation is similar to Port-1.
- Port-2 latch remains stable when Port-2 pin are used for external memory access.

PORT-3

- Port-3 (P3.0-P3.7) having alternate functions to each pin. The internal structure of a port-3 pin is shown in fig below.



- Each pin of Port-3 can be individually programmed for I/O operation or for alternate function.
- The alternate function can be activated only if the corresponding latch has been written to '1'.
- To use the port as input port, '1' should be written to the latch.
- Alternate functions of Port-3 pins:

P3.0	RxD
P3.1	TxD
P3.2	$\overline{\text{INT0}}$
P3.3	$\overline{\text{INT1}}$
P3.4	T0
P3.5	T1
P3.6	$\overline{\text{WR}}$
P3.7	$\overline{\text{RD}}$

3.6.5 REGISTER ORGANIZATION

The 8051's on-chip memory consists of 256 memory bytes organised as follows:

<i>First 128 bytes:</i>	00h to 1Fh	Register Banks
	20h to 2Fh	Bit Addressable RAM
	30 to 7Fh	General Purpose RAM
<i>Next 128 bytes:</i>	80h to FFh	Special Function Registers

Internal RAM:

The first 128 bytes of internal memory is organised as shown in figure, and is referred to as Internal RAM.

1. Register banks:

- o 32 bytes from address 00h to 1Fh that make up 32 working registers organized as 4 memory banks of 8 registers each.
- o The 4 register banks are numbered 0 to 3 and are made up of 8 registers named R0 to R7. Each register can be addressed by name or by its RAM addresses. Thus R0 of bank3 is R0 (if bank3 is selected) or address 18h (where bank3 is selected).
- o Bits RS0 and RS1 in the PSW determine which bank of registers is currently in use at any time when program is running.
- o Register banks not selected can be used as general purpose RAM.
- o Bank0 is selected by default on reset.

2. Bit addressable memory:

- o A bit addressable area of 16 bytes occupies RAM byte addresses 20h to 2Fh, forming total of 128 bits.
- o This is where individual memory bits in Internal RAM can be set or cleared.
- o A bit variable can be set with a command such as SETB and cleared with a command such as CLR. Example instructions are:

SETB 25h ; sets the bit with address 25h (becomes 1)

CLR 25h ; clears bit with address 25h (becomes 0)

3. A general purpose RAM:

- o A general purpose RAM area above the bit area from 30h to 7Fh, addressable as byte.
- o These 80 bytes of Internal RAM memory are available for general-purpose data storage.
- o The general purpose RAM can be accessed using direct or indirect addressing modes.

Examples of direct and indirect addressing:

MOV A, 6Ah	; reads contents of address 6Ah to accumulator
MOV A, @R1	; move indirect: R1 contains address of Internal RAM which contains data that is moved to A.

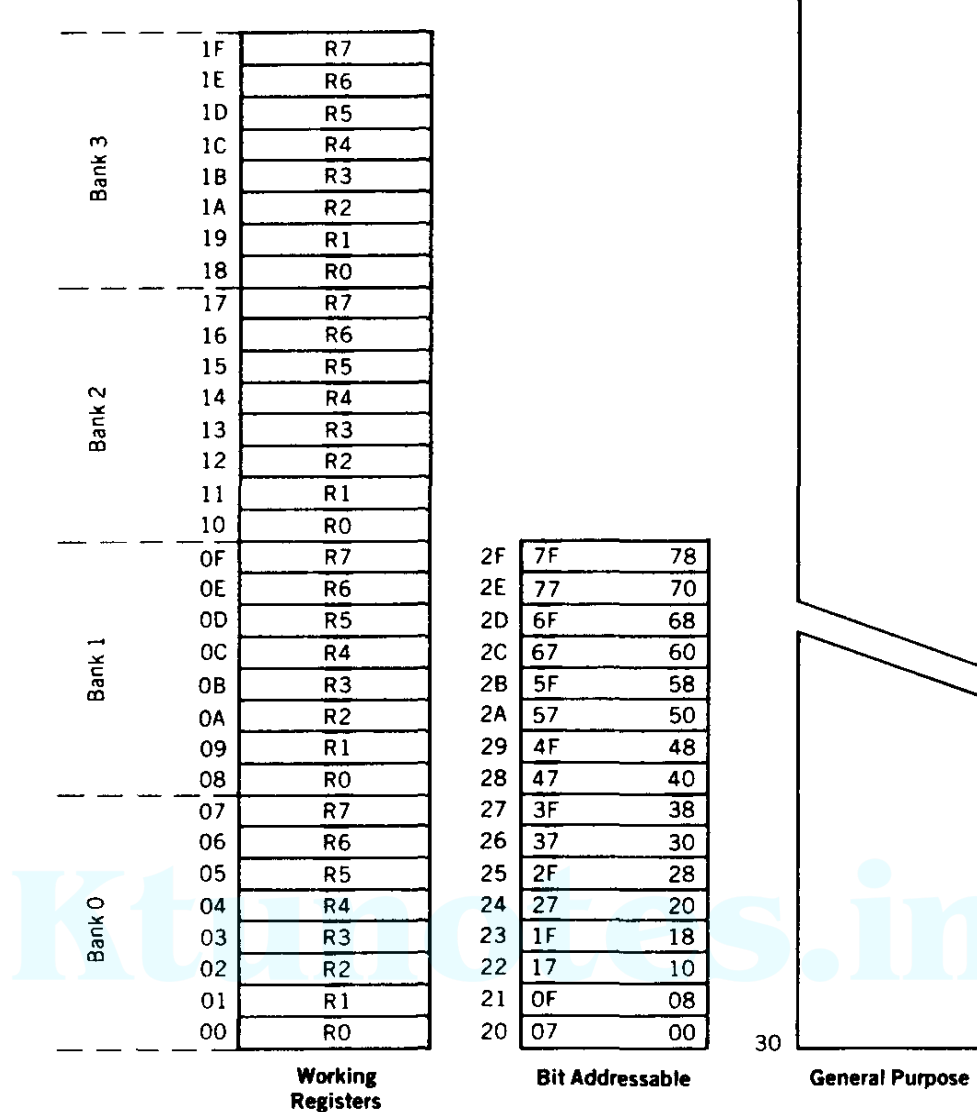


Fig. Internal RAM structure

Special Function Registers (SFR):

- Special function registers (SFR) are addressed using addresses from 80h to 0FFh.
- Some SFRs are also bit addressable as is the case for the bit area of RAM.
- Not all of the addresses from 80h to 0FFh are used for SFRs. Only the addressed ones can be used in programming SFRs and equivalent internal RAM addresses are shown in Figure below
- The set of Special Function Registers (SFRs) contain important registers such as Accumulator, Register B, and I/O Port latch registers, Stack pointer, Data Pointer, Processor Status Word (PSW) and various control registers.
- Some of these registers are bit addressable (they are marked with a * in the Figure below).

ADDRESS	Special Function Registers							
0F8H								
0F0H	B*							
0E8H								
0E0H	ACC*							
0D8H								
0D0H	PSW*							
0C8H	(T2CON)*		(RCAP2L)	(RCAP2H)	(TL2)	(TH2)		
0C0H								
0B8H	IP*							
0B0H	P3*							
0A8H	IE*							
0A0H	P2*							
98H	SCON*	SBUF						
90H	P1*							
88H	TCON*	TMOD	TL0	TL1	TH0	TH1		
	P0*	SP	DPL	DPH				PCON
80H	80H	81	82	83	84	85	86	87

